

TASK - 3

Q - If array is sorted check.

Input $\rightarrow$ {90, 80, 100, 70, 90, 30}

Output $\rightarrow$ false

```
int flag = 1;
```

```
for (int i = 0; i < arr.size() - 1; i++)
```

```
{
```

```
    if (arr[i] > arr[i+1])
```

```
{
```

```
        flag = 0;
```

```
        break;
```

```
}
```

```
}
```

```
return flag;
```

dry run

90, 80, 100, 70, 90, 30

↑

i: 90 > 80 ✓

flag = 0 ✓

false. Not sorted.

Q - find nos with even no of digits.

Input $\rightarrow$ [12, 245, 6, 2, 7896]

Output $\rightarrow$ 2.

```
int count = 0;
```

```
int i;
```

```
for (i = 0; i < arr.size(); i++)
```

```
{
    int n = arr[i];
```

```
    int digit = 0;
```

```

while (n > 0) {
    digits = n % 10
    n = n / 10
    if (digits % 2 == 0) {
        count++;
    }
    n = n / 10;

```

```

if (count % 2 == 0) {
    digit++;
}

```

```

if (digits % 2 == 0) {
    count++;
}

```

↘

```

return count;

```

Dry run:

$i = 0 \rightarrow 12 \begin{matrix} \swarrow 1 \\ \searrow 2 \end{matrix}$, so 2 digits $\rightarrow$ even $\rightarrow$ count = 1

$i = 1 \rightarrow 345 \begin{matrix} \swarrow 3 \\ \searrow 4 \\ \quad \searrow 5 \end{matrix}$, so 3 digits $\rightarrow$ odd

$i = 2 \rightarrow 6 \rightarrow$ so 1 digit $\rightarrow$ odd

$i = 3 \rightarrow 2 \rightarrow$ so 1 digit $\rightarrow$ odd

$i = 4 \rightarrow 7896 \rightarrow$ so 4 digits $\rightarrow$ even $\rightarrow$ count = 2

Count = 2 Ans.

Q- Search Insert Position : $\text{input} \rightarrow [1, 3, 5, 6]$, $\text{target} = 2$
 $\text{output} \rightarrow 2$

```
int start, mid, end;
int index = 0;
```

```
int n = array size();
```

```
start = 0;
```

```
end = n - 1;
```

```
while (start <= end) {
```

```
    mid = start + (end - start) / 2;
```

```
    if (arr[mid] == target)
```

```
        index = mid;
```

```
        break;
```

```
    else if (arr[mid] > target)
```

```
        end = mid - 1;
```

```
    else
```

```
        start = mid + 1;
```

```
}
```

```
if (index == 0)
```

```
    return start;
```

```
else
```

```
    return mid;
```

Dry run

$\begin{matrix} 0 & 1 & 2 & 3 \\ 1 & 3 & 5 & 6 \end{matrix}$, $\text{target} = 2$

$\text{mid} = \frac{0 + 3}{2} = 1 \Rightarrow 3 < 5$ $\text{start} = 2$

$\text{end} = 3 \Rightarrow 1 \text{ Ans.}$

$\frac{1 + 3}{2} = 2$

$5 = 2 \times$

Q- first Bad version. ^{input}
 $n = 5$ ^{output}
 $bad = 4$

```
int start, mid, end;
```

```
int answer;
```

```
start = 1;
```

```
end = n;
```

```
while (start <= end) {
```

```
    mid = start + (end - start) / 2;
```

```
    if (isBadVersion(mid)) {
```

```
        answer = mid;
```

```
        end = mid - 1;
```

```
    }
```

```
    else
```

```
        start = mid + 1;
```

```
    }
```

```
    return answer;
```

Dry run :

$start = 1, end = 5$

$1 \leq 5 \checkmark$

$1 + 2 \neq 3 \rightarrow \text{false}$

$start = 2, end = 5$

$2 \leq 5$

$2 + 3 (2 \neq 3) - \text{false}$

$3 + 5 = 3/2$

$n = 4$

$4 + 1 = \text{true}$

Answer = 4

Q - Sqrt(x)

Input $\rightarrow x = 9$ Output $\rightarrow 3$

```
int start, mid, end;
int ans;
```

```
start = 1;
```

```
end = x;
```

```
if (x < 2)
```

```
{
```

```
    return x;
```

```
}
```

```
while (start <= end) {
```

```
    mid = start + (end - start) / 2;
```

```
    if (mid == x / mid)
```

```
        ans = mid;
```

```
        break;
```

```
    else if (mid < x / mid)
```

```
        ans = mid
```

```
        start = mid + 1;
```

```
    else
```

```
        end = mid - 1;
```

```
}
```

```
return ans;
```

```
;
```

Dry run:

$$x = 8$$

$$\text{start} = 1$$

$$\text{end} = 8$$

$$1 < 8$$

$$\text{mid} = 1 + (8 - 1) / 2$$

$$\text{mid} = 4$$

$$4 == 8 / 4$$

$$4 == 2 \quad \times$$

$$4 < 2 \quad \times$$

$$4 > 2 \quad \checkmark$$

$$\text{end} = \text{mid} - 1$$

$$= 3$$

$$\text{start} = 1$$

$$\text{end} = 3$$

$$1 < 3 \quad \checkmark$$

$$\text{mid} = 1 + (3 - 1) / 2$$

$$\text{mid} = 2$$

$$2 == 8 / 2$$

$$2 == 4 \quad \times$$

$$2 < 4 \quad \checkmark$$

$$\text{ans} = 2 \text{ maybe bounds } \text{so, } 3 == 8 / 3$$

$$\text{start} = \text{mid} + 1 \Rightarrow \text{start} = 3, \text{end} = 3 \quad 3 == 2 \quad \times$$

$$\text{So, ans} = 2 \text{ Ans.}$$

$$3 > 2$$

$$\text{end} = \text{mid} - 1$$

$$\text{so,}$$

$$\text{end} = -1$$

$$3 < 3 \quad \checkmark$$

$$\text{out of}$$

$$\text{mid} = 3$$

Input $\rightarrow$

5	7	7	8	5	10
---	---	---	---	---	----

, target = 8

```
start = mid + 1;
```

PAGE No.:

DATE:

else if (arr[mid] < target)
start = mid + 1;

else

end = mid - 1;

↳

return (first, last)

Tricky Code Analysis

Case 1 Binary Search on Unsorted Array:

```

public class Test2 {
    psum (String args []) {
        int [] arr = {7, 2, 9, 4, 1};
        int target = 4;
        int low = 0, high = arr.length - 1,
            ans = -1;
        while (low <= high) {
            int mid = (low + high) / 2;
            int if (arr[mid] == target) {
                ans = mid;
                break;
            }
            else if (arr[mid] < target)
                low = mid + 1;
            else
                high = mid - 1;
        }
        System.out.println (ans);
    }
}

```

Ans: as the array is unsorted so, the ans = -1 as it cannot find the value.

for binary search the array should be sorted.

Case 2 The Mid Overflow Trap

```

public class Test2 {
    psvm (String args[]) {
        int low = 2000000000;
        int high = 2100000000;

        println ((low + high) / 2);
        println (low + (high - low) / 2);
    }
}

```

Output

Overflow

2050000000

Case 3 The Stuck Pointer

```

public class Test3 {
    psvm (String args[]) {
        int arr = {1, 3, 5};
        int target = 5;
        int low = 0, high = arr.length - 1;
        while (low < high) {
            int mid = (low + high) / 2;
            if (arr[mid] < target)
                low = mid;
            else
                high = mid;
        }
        println (low);
    }
}

```

arr = [1, 3, 5]

target = 5

start = 0

end = 2

mid = $\frac{0+2}{2} = 1$

3 < 5 ✓

~~low~~ start = 1

start = 1

end = 2

mid = $\frac{1+2}{2} = 1$

3 < 5

start = 1

start = 1

end = 2

mid = $\frac{1+2}{2} = 1$

Ans Error: the statements are not cleared and defined

it should be

while (low <= high) {

mid = low + (high - low) / 2;

if (arr[mid] == target) {

```

    repln (mid);
    break;
else if (arr[mid] < target)
    low = mid + 1;
else
    high = mid - 1;

```

Output
Infinite loop

Ques 4: Built in Binary Search Return Values

```
import java.util. Arrays;
```

```

public class Test4 {
    psum (String args[]) {
        int arr[] = {10, 20, 30, 40};
        repln (Arrays.binarySearch (arr, 40));
        repln (Arrays.binarySearch (arr, 25));
        repln (Arrays.binarySearch (arr, 5));
    }
}

```

Output

3

Ans: as if target not matched then
this function returns

-3

-1

- (insertion point) - 1

for 25 → 4 - 2 - 1 = -3

for 5 → 4 - 0 - 1 = -1

Ques 5: Sorting Strings (Capital vs Small)

Input java.util. Array;

```
public class Test5 {
    main (String args[]) {
        String [] fruits = {"banana", "apple", "Orange"};
        Arrays.sort (fruits);
        println (Arrays.toString (fruits));
    }
}
```

Output

Cherry

apple

banana

Case 6: Linear Search Without break

```
public class Test6 {
    main (String args[]) {
        int [] arr = {4, 5, 1, 2};
        int idx = -1;

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == 4) {
                idx = i;
            }
        }

        println (idx);
    }
}
```

idx = -1 $\frac{0+1+2}{3} = 1$

i = 0, 1, 2, 3

arr[0] == 4

4 == 4 ✓

idx = i; & idx = 0

arr[1] == 4

5 == 4 ✗

~~idx~~

arr[2] == 4

4 == 4 ✓

idx = 2

arr[3] == 4

2 == 4 ✗

Output

2

There should be break because if 4 is not present other than at 0 index then it is a waste to check. The loop should break once it already found the answer.

Ques 7 Recursive Binary Search

```
public class Test 7 {
```

```
    public int search (int arr[], int lo, int hi, int t) {
```

```
        static int search (int arr[], int lo, int hi, int t) {
```

```

if (lo > hi)
    return -1;
int mid = (lo + hi) / 2;

```

```

if (a[mid] == t)
    return mid;

```

```

if (a[mid] < t)
    search(a, mid + 1, hi, t);
else
    search(a, lo, mid - 1, t);
}

```

Output

Ans The error is return statement is not there when function is called so it will not return the function value.

Case 8: Wrong Starting value of high

```

public class Test9 {
    public static void main(String args[]) {
        int arr[] = {1, 3, 5};
        int target = 9;
        int low = 0, high = arr.length;
    }
}

```

```

while (low <= high) {

```

```
int mid = (low + high) / 2;
```

```
if (arr[mid] == target)
```

```
    println(found);
```

```
else if (arr[mid] < target)
```

```
    low = mid + 1;
```

```
else
```

```
    high = mid - 1;
```

```
{
```

```
{
```

```
}
```

Output

Index out of bounds

Reason: High value is initialized wrong.

It should be $High = arr.length - 1;$

Ques 9: Comparing Wrapper Objects

```
public class Test9 {
```

```
    public static void main (String args[]) {
```

```
        Integer a = 127, b = 127;
```

```
        Integer c = 1000, d = 1000;
```

```
        println(a == b);
```

```
        println(c == d);
```

```
        println(c.equals(d));
```

```
{
```

```
{
```

Ans

Output:

true

false

true

Ques 10

Searching in ArrayList

import java.util. ArrayList;

import java.util. Arrays;

public class Test 64

prgm {String args []}

ArrayList < Integer > list = new ArrayList
< > (Array.asList
(5, 10, 15, 20));

System.out.println (list.indexOf (10));

System.out.println (list.^{last}IndexOf (10));

System.out.println (list.indexOf (99));

~~System.out.println~~ list.contains (Integer.valueOf (10));

System.out.println (list.indexOf (10));

}

}

Ans

Output

1

3

-1

2

Code Modification Challenge :

In VS Code.

2nd
part

for target = 18

Linear Search

Binary Search

1) $i = 0$
 $2 == 18 \times$ 2) $i = 1$
 $4 == 18 \times$ 3) $i = 2$
 $9 == 18 \times$ 4) $i = 3$
 $16 == 18 \times$ 5) $i = 4$
 $25 == 18 \times$ 6) $i = 5$
 $36 == 18 \times$ 7) $i = 6$
 $49 == 18 \times$ 8) $i = 7$
 $64 == 18 \times$ 9) $i = 8$
 $81 == 18 \times$

Total : 9 times

 $O(n)$ 1) $start = 0$
 $end = 9$ $mid = 4$ $16 == 18 \times$ 2) $start = 5$
 $end = 9$ $mid = 7$ $49 == 18 \times$ 3) $start = 8$
 $end = 9$ $mid = 8$ $81 == 18 \checkmark$

Total : 3 times

 $O(\log n)$